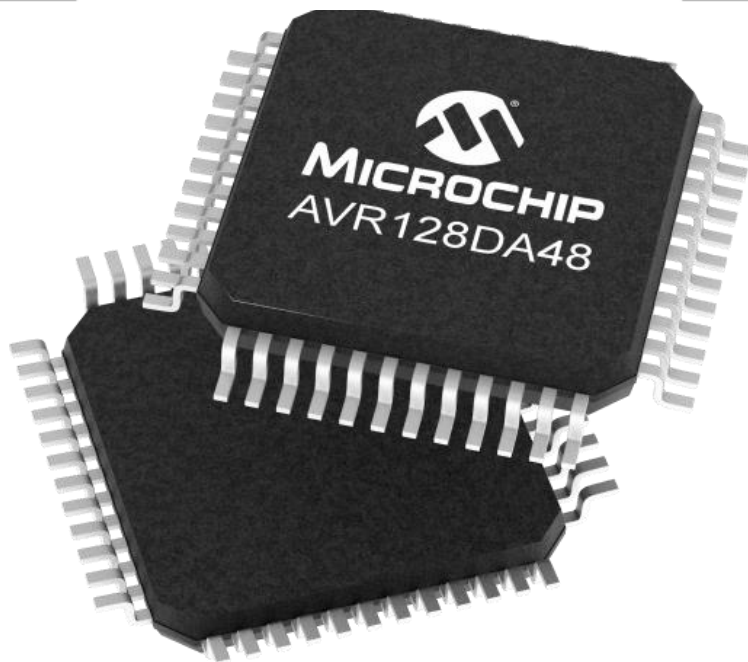
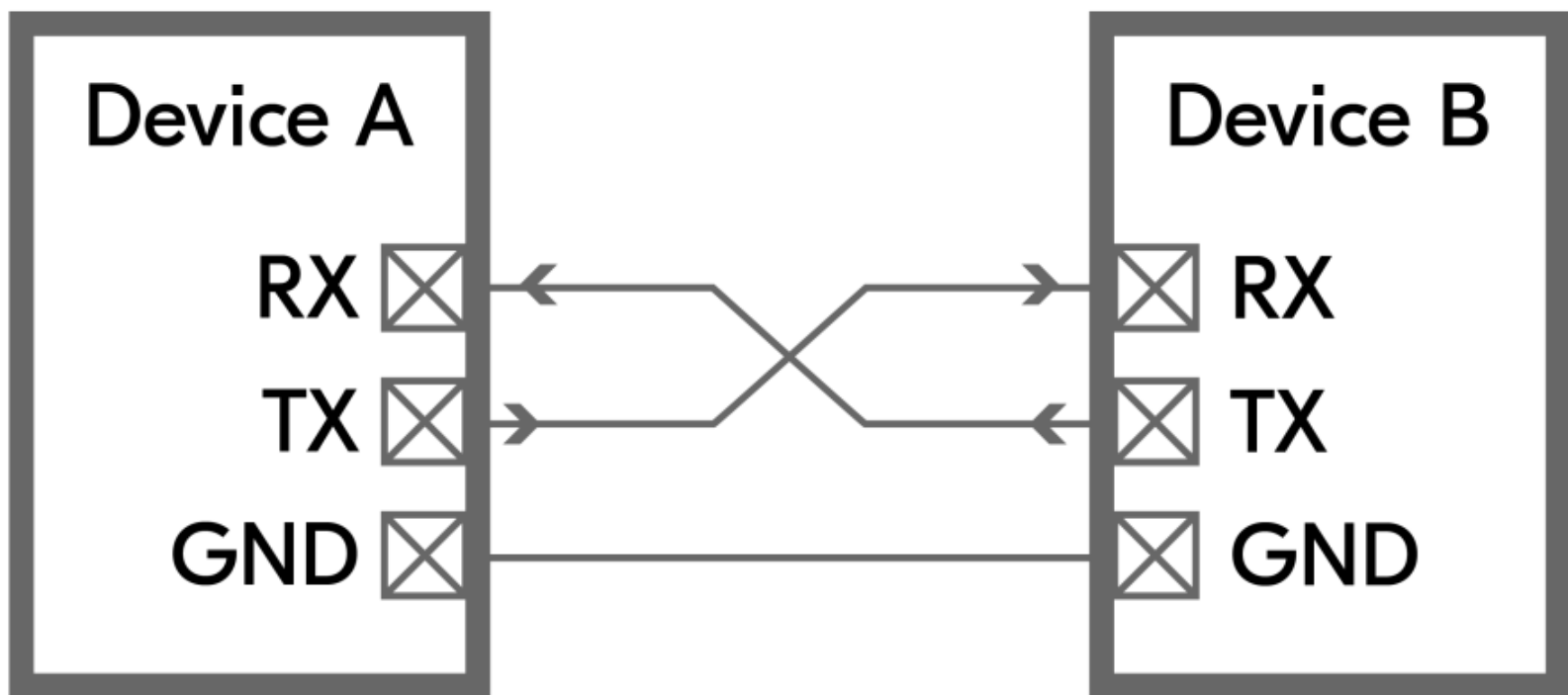


“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 11 - USART Fundamentals



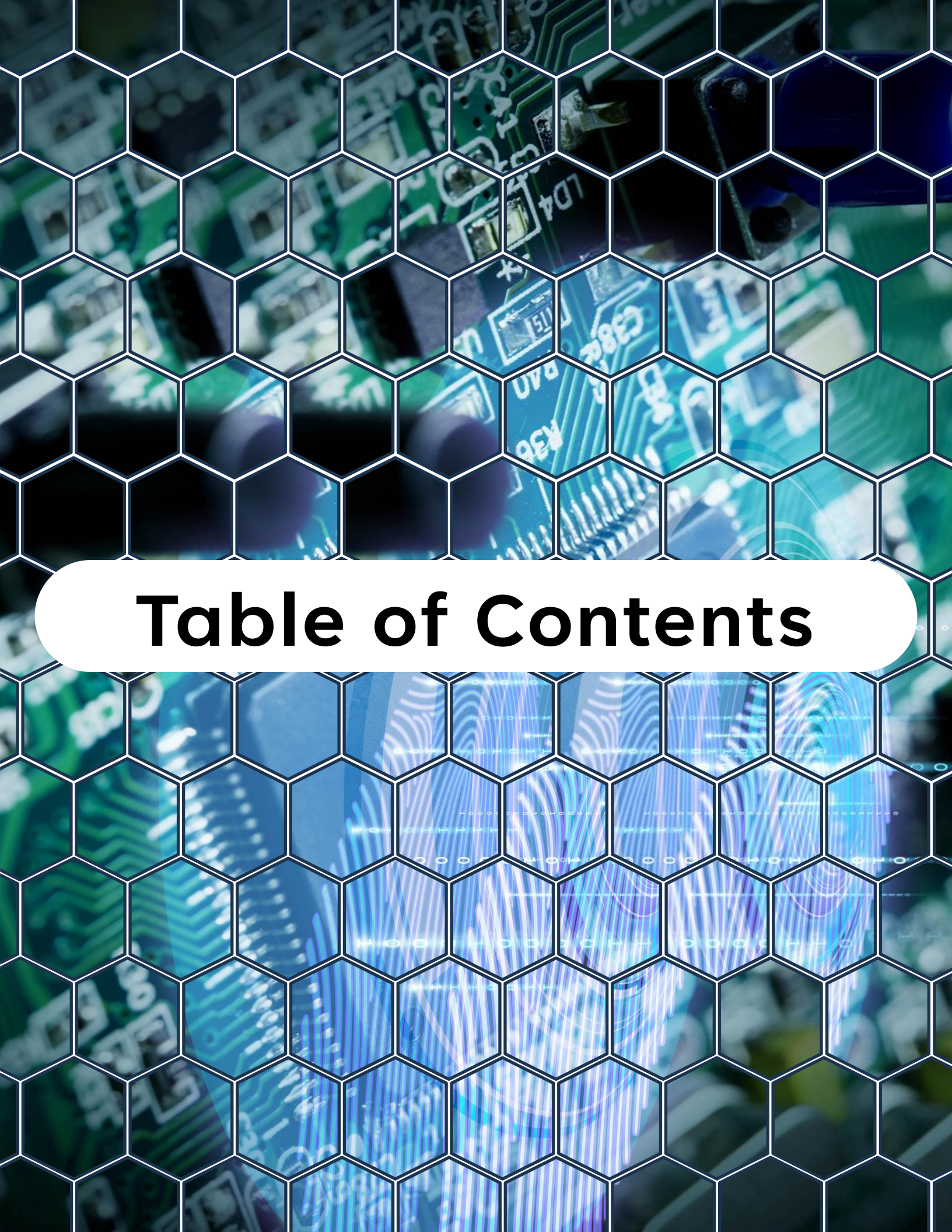
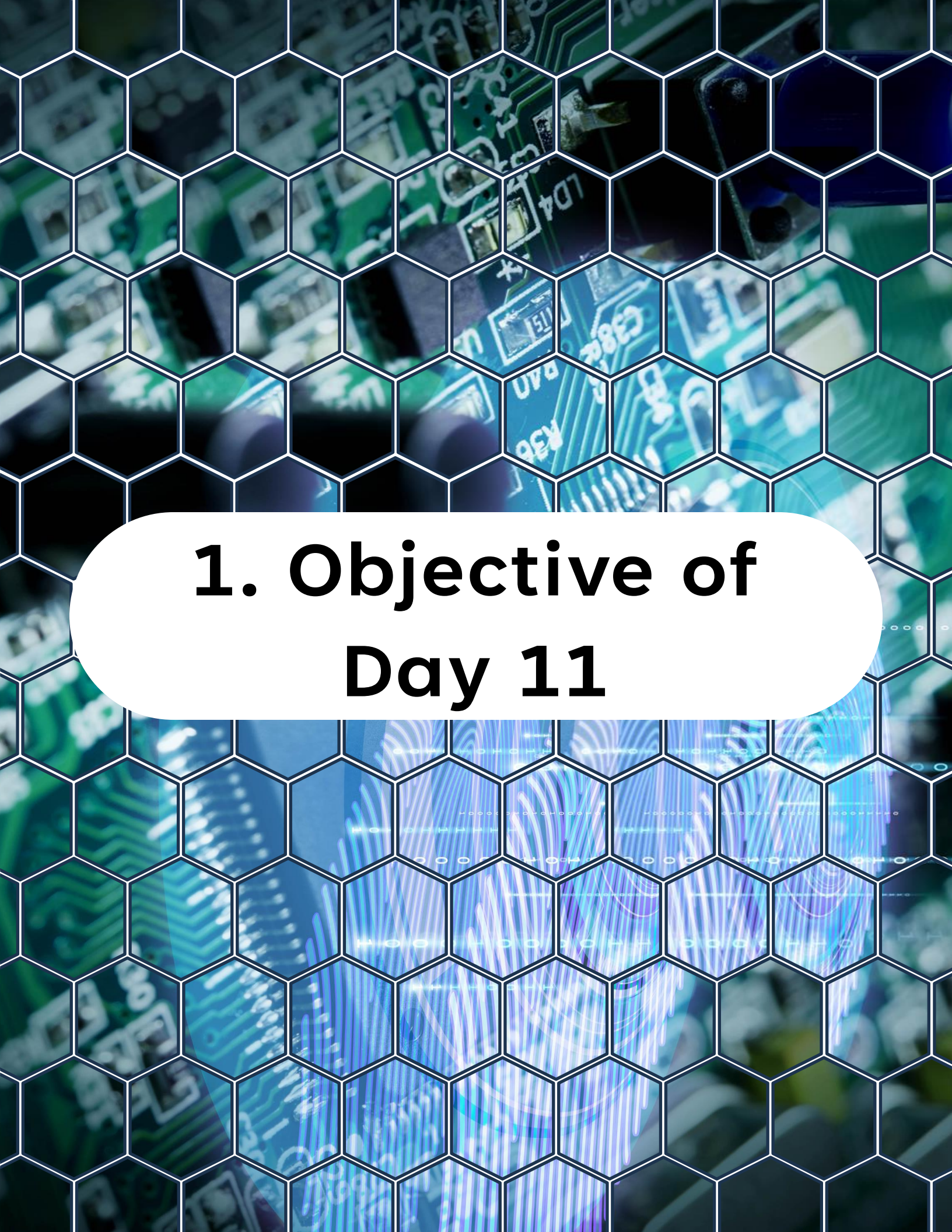


Table of Contents

Table of Contents

1. Objective of Day 11
2. What is USART?
3. Where USART Fits in AVR128DA48
4. Serial Communication Basics
5. Baud Rate and Why It Matters
6. USART Clocking and Baud Calculation
7. Basic USART Initialization (Polling Mode)
8. Transmitting One Byte (Polling)
9. Transmitting a String
10. Receiving One Byte (Polling)
11. Minimal main() Example
12. Debugging USART Problems
13. Common USART Mistakes
14. What You Learned Today
15. What Comes Next



1. Objective of Day 11

1. Objective of Day 11

Today your MCU starts talking.

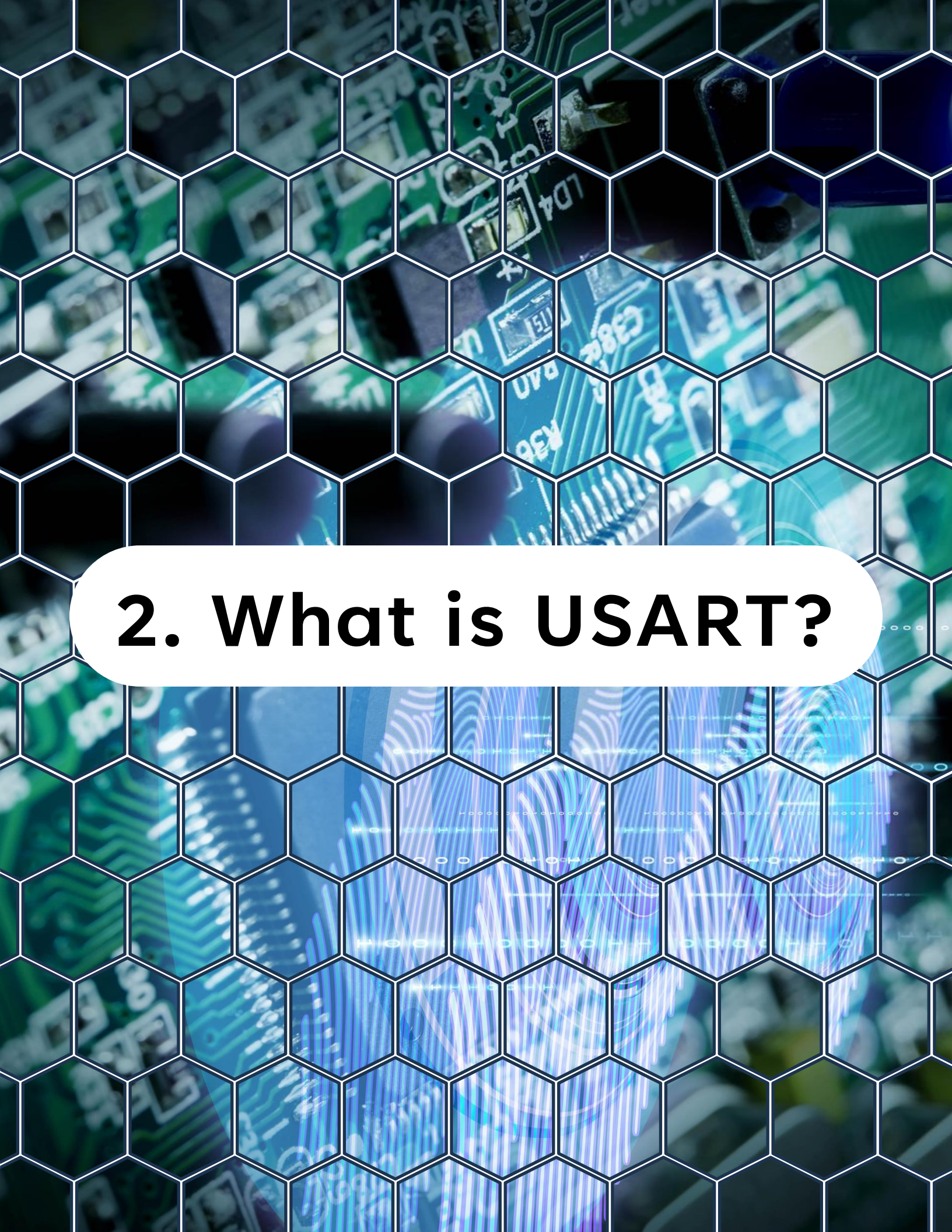
You will configure the **USART** peripheral at register level and transmit real serial data to a PC terminal.

No Arduino Serial.print. No middleware. Just registers and math.

By the end of this day, you will:

- Understand asynchronous serial communication
- Configure **USART** for TX and RX
- Calculate baud rate correctly
- Transmit characters and strings
- Receive data using polling
- Debug common UART mistakes quickly

This is a major milestone. Communication changes everything.



2. What is USART?

2. What is USART?

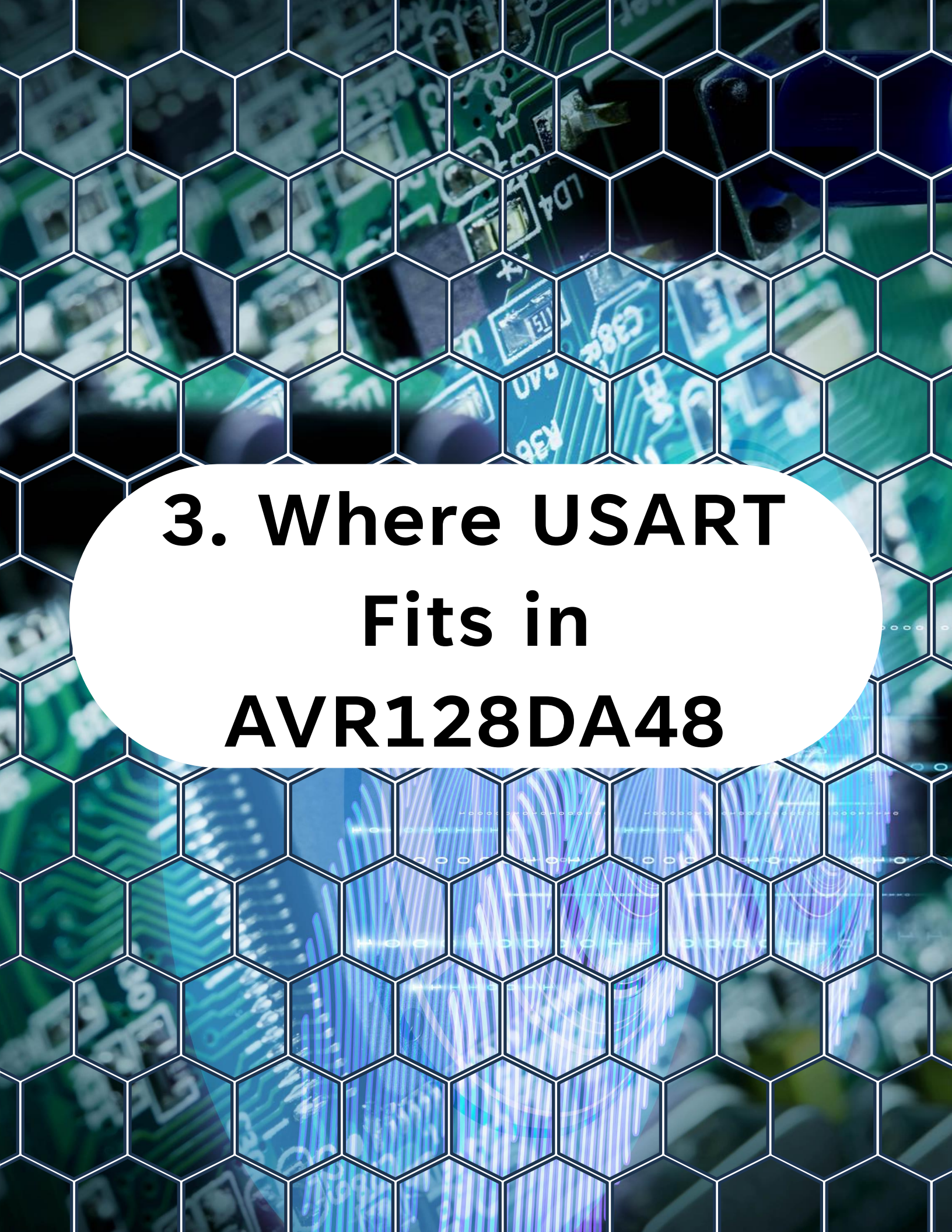
USART stands for **Universal Synchronous and Asynchronous Receiver Transmitter**.

It is simple, fast, robust, and widely supported communication peripheral.

In this course, we focus on asynchronous mode, commonly called UART.

Asynchronous serial communication uses:

- One **TX** line
- One **RX** line
- One **XDIR** line (Transmit enable for RS-485)
- A shared baud rate (Clock for synchronous operation)
- No clock line



3. Where USART Fits in AVR128DA48

3. Where USART Fits in AVR128DA48

The AVR128DA48 contains multiple USART peripherals (USART0, USART1, etc.).

Each USART provides:

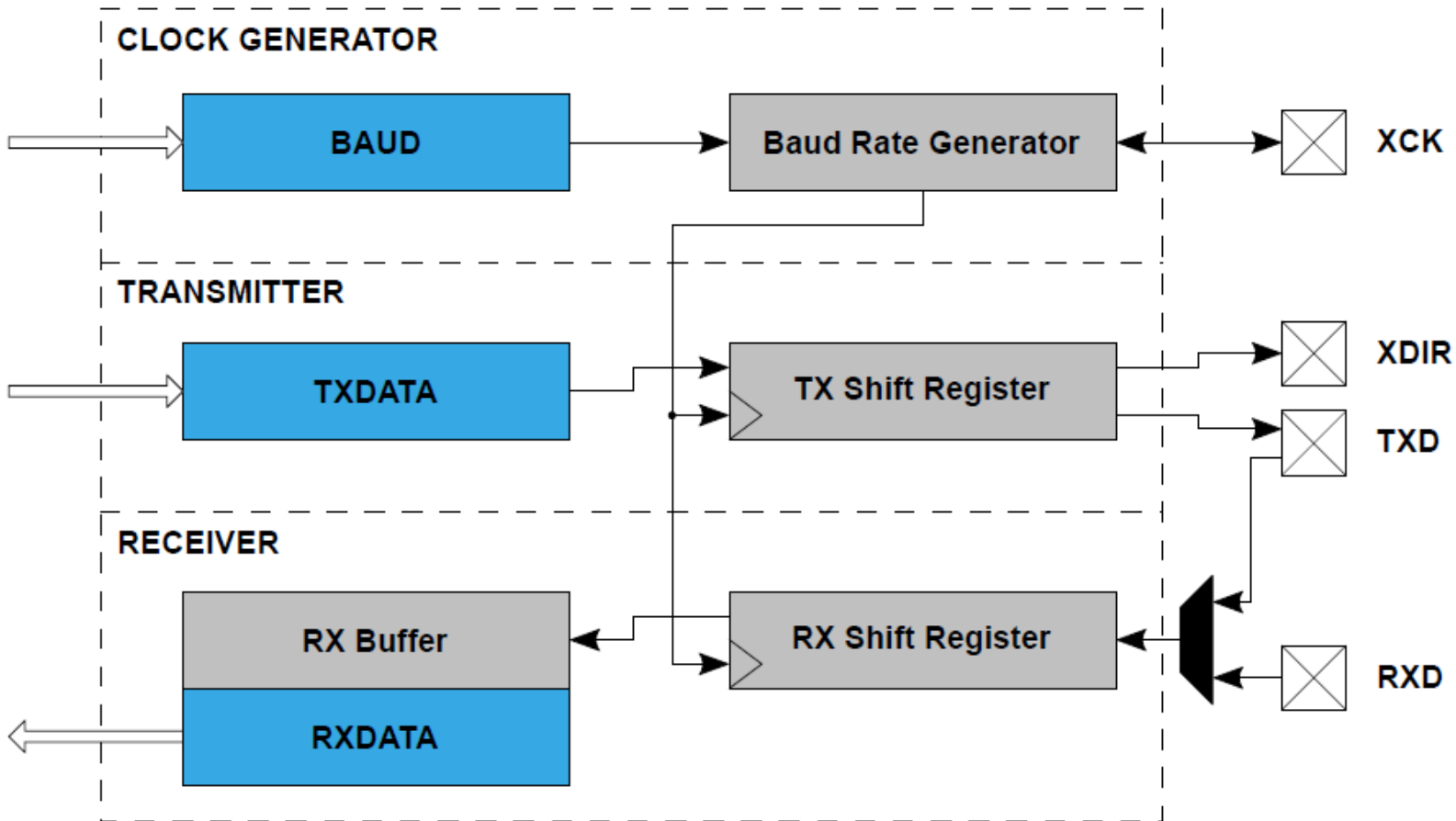
- TX (transmit)
- RX (receive)
- Baud rate generator
- Status flags
- Interrupt capability

Today you will start with polling mode.

Interrupt-driven communication comes later.

3. Where USART Fits in AVR128DA48

USART Block Diagram



USART Signal Description

Signal	Type	Description
XCK	Output/input	Clock for synchronous operation
XDIR	Output	Transmit enable for RS-485
TxD	Output/input	Transmitting line (and receiving line in One-Wire mode)
RxD	Input	Receiving line



4. Serial Communication Basics

4. Serial Communication Basics

A UART frame typically consists of:

- 1 start bit (low)
- 8 data bits
- Optional parity
- 1 stop bit (high)

Common configuration:

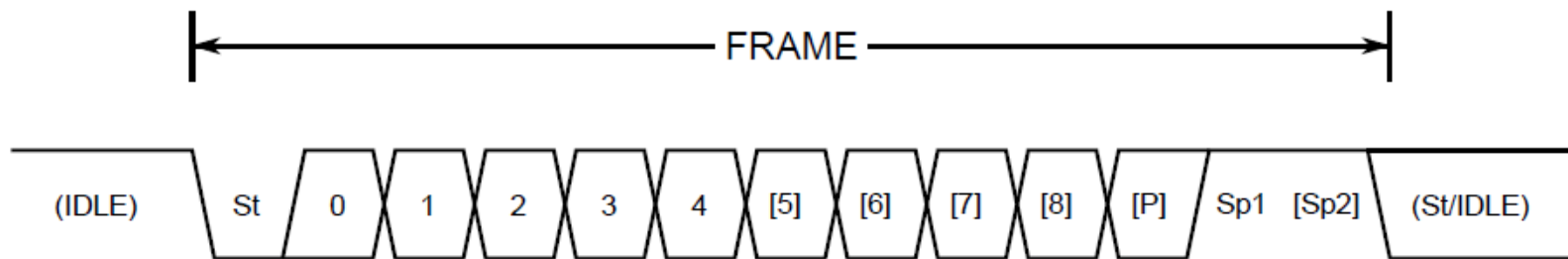
- 9600 baud
- 8 data bits
- No parity
- 1 stop bit

This is often called **8N1**.

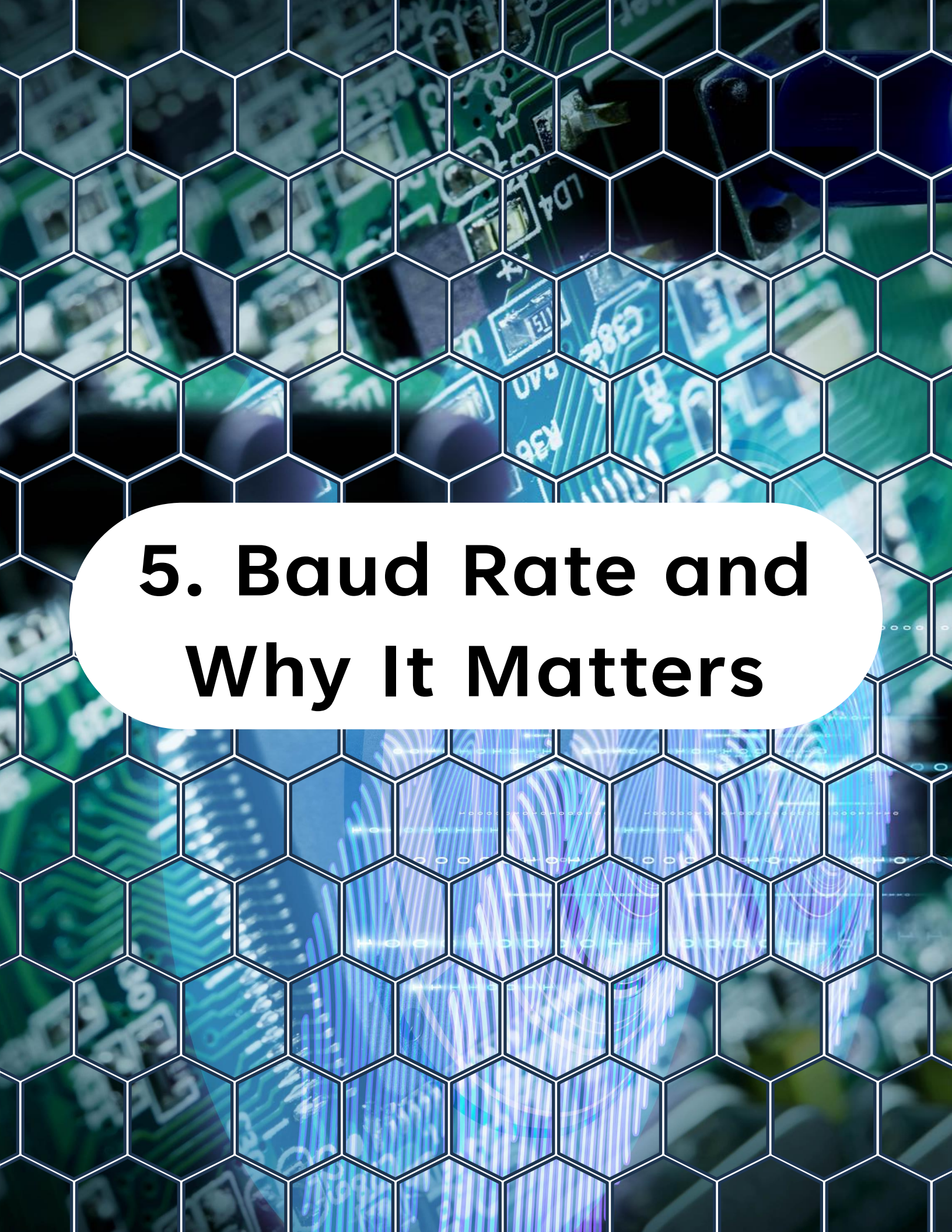
4. Serial Communication Basics

Frame Formats

This figure illustrates the possible combinations of frame formats. Bits inside brackets are optional.



St	Start bit, always low
(n)	Data bits (0 to 8)
P	Parity bit, may be odd or even
Sp	Stop bit, always high
IDLE	No transfer on the communication line (RxD or TxD). The Idle state is always high.



5. Baud Rate and Why It Matters

5. Baud Rate and Why It Matters

Baud rate defines how fast bits are transmitted.

Common values:

- 9600
- 19200
- 38400
- 115200

Both sides must agree on baud rate. If not:

- You see garbage characters
- Framing errors occur
- Communication fails

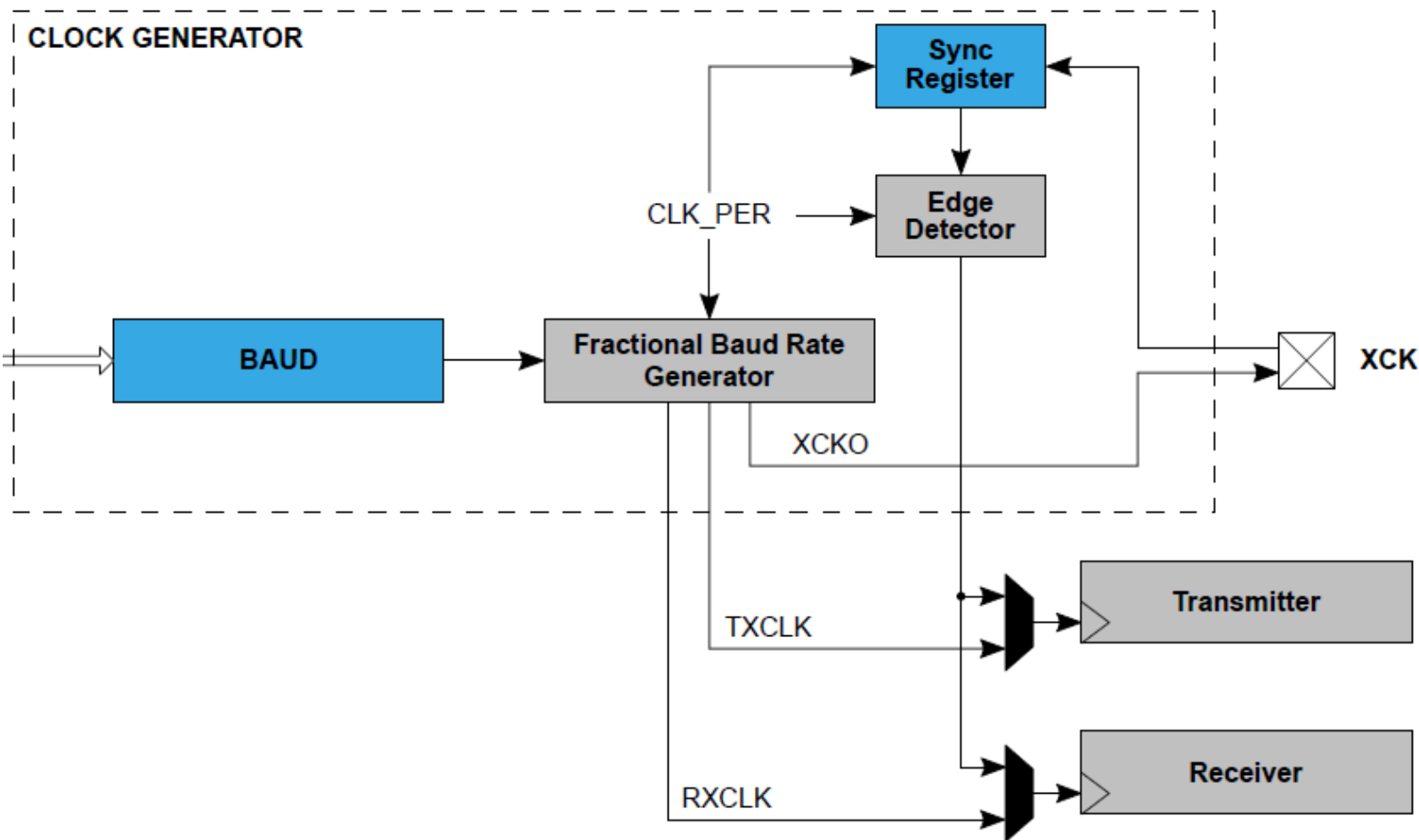
Baud rate depends on:

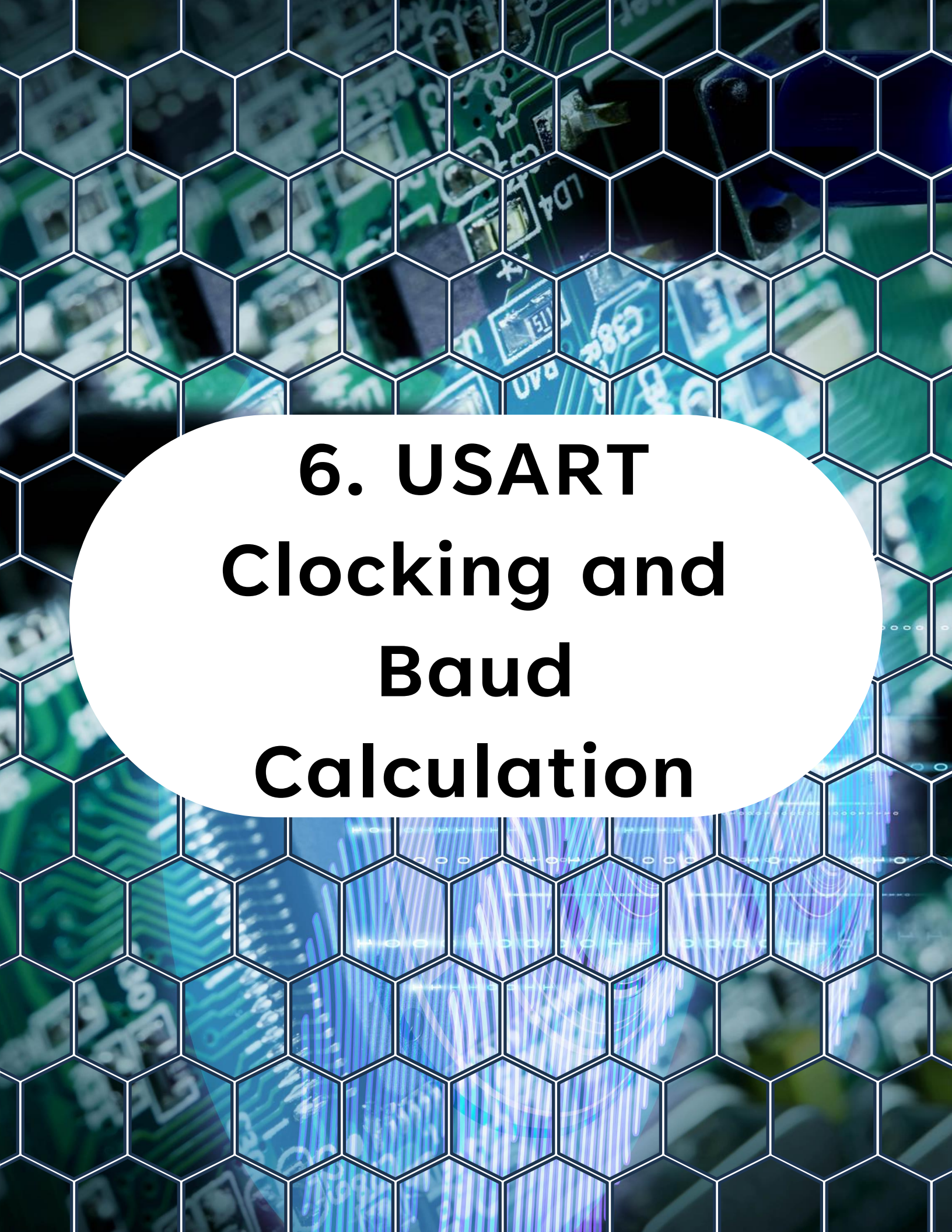
- System clock (Day 3)
- USART baud register value

5. Baud Rate and Why It Matters

If your clock is wrong, your serial communication is wrong.

Clock Generation Logic Block Diagram





6. USART Clocking and Baud Calculation

6. USART Clocking and Baud Calculation

USART uses the peripheral clock **CLK_PER** register.

In asynchronous normal mode, a simplified formula is:

$$\text{BAUD} = (\text{F_CLK_PER} / (16 * \text{baud_rate}))$$

On AVR DA series, the exact register formula differs slightly due to fractional scaling. The datasheet provides the correct equation.

For simplicity in this course, we will assume:

- $\text{F_CLK_PER} = 24 \text{ MHz}$
- Target baud = 9600

6. USART Clocking and Baud Calculation

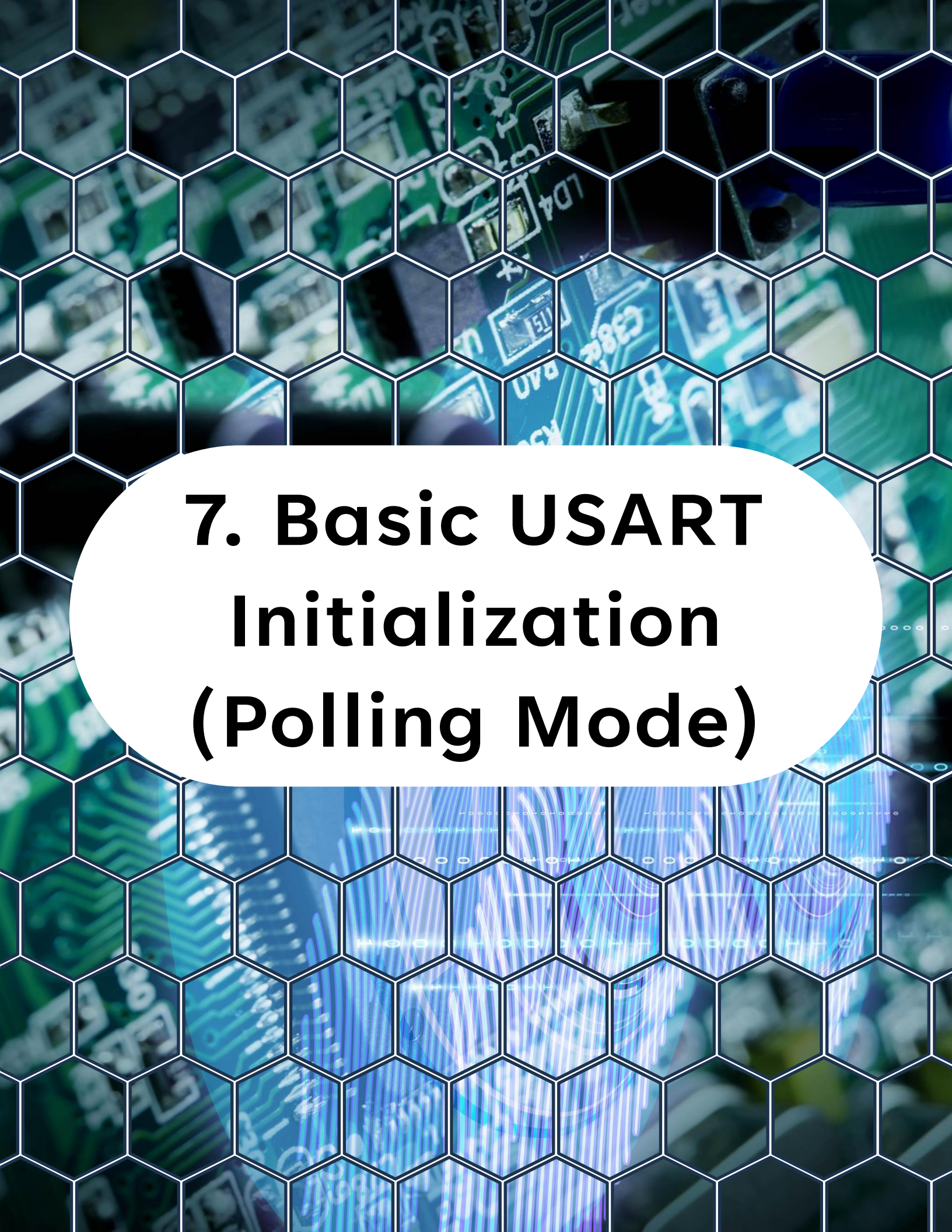
The IDE can help calculate exact **BAUD** register values, but you must understand the relationship.

Equations for Calculating Baud Rate Register Setting

Operating Mode	Conditions	Baud Rate (Bits Per Seconds)	USART.BAUD Register Value Calculation
Asynchronous	$f_{BAUD} \leq \frac{f_{CLK_PER}}{S}$ $USART.BAUD \geq 64$	$f_{BAUD} = \frac{64 \times f_{CLK_PER}}{S \times BAUD}$	$BAUD = \frac{64 \times f_{CLK_PER}}{S \times f_{BAUD}}$
Synchronous Master	$f_{BAUD} \leq \frac{f_{CLK_PER}}{S}$ $USART.BAUD \geq 64$	$f_{BAUD} = \frac{f_{CLK_PER}}{S \times BAUD[15:6]}$	$BAUD[15:6] = \frac{f_{CLK_PER}}{S \times f_{BAUD}}$

S is the number of samples per bit

- Asynchronous Normal mode: $S = 16$
- Asynchronous Double-Speed mode: $S = 8$
- Synchronous mode: $S = 2$



7. Basic USART Initialization (Polling Mode)

7. Basic USART Initialization (Polling Mode)

We will configure:

- 9600 baud
- 8 data bits
- No parity
- 1 stop bit
- TX and RX enabled

Example: USART0 Initialization

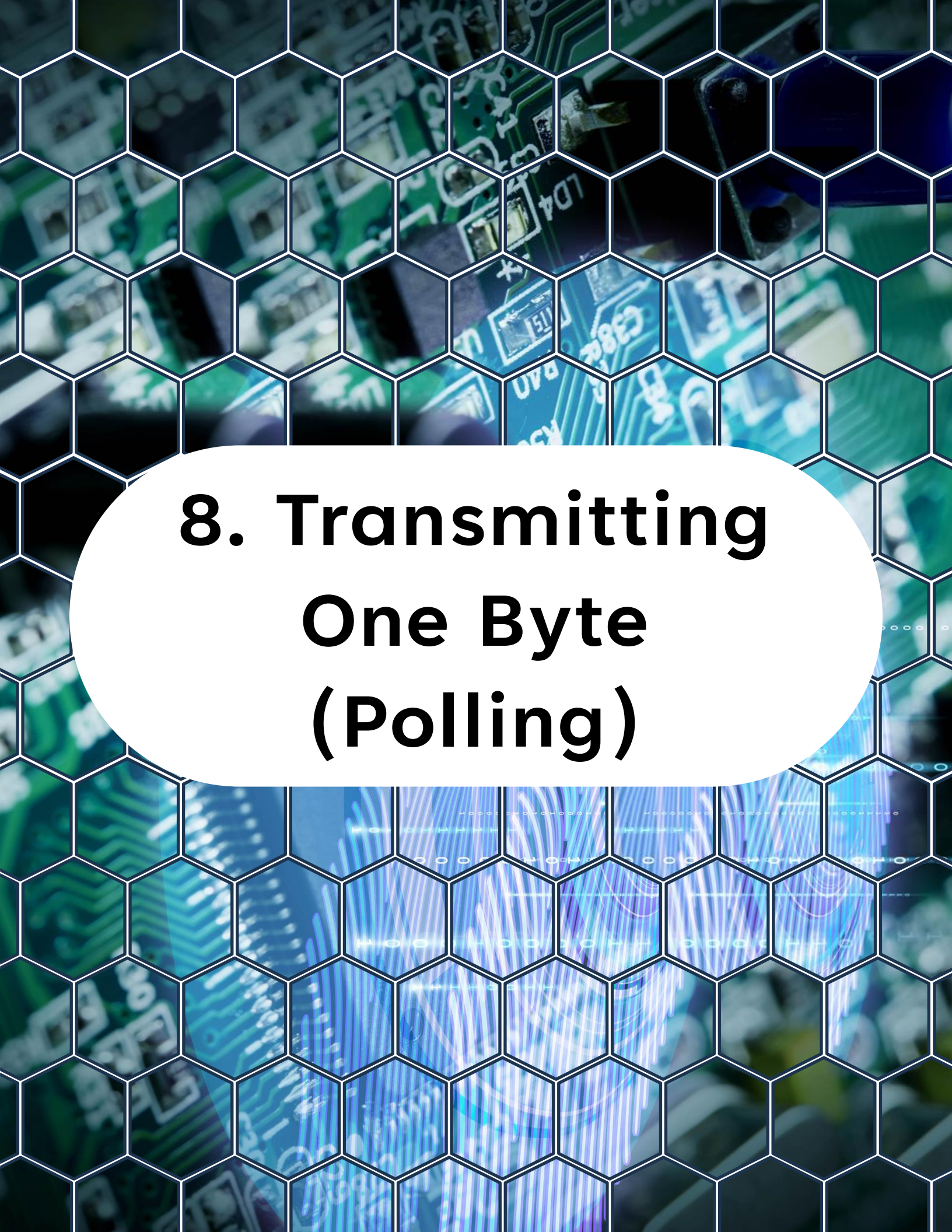
```
1  #include <avr/io.h>
2
3  #define F_CPU_HZ    (24000000UL)
4  #define BAUD_RATE   (9600UL)
5
6  /*
7   * BAUD calculation for AVR DA series (asynchronous normal mode)
8   * Formula from datasheet:
9   * BAUD = (F_CPU * 64) / (16 * baud_rate)
10  */
11 #define USART0_BAUD_VALUE ((uint16_t)((F_CPU_HZ * 64UL) / (16UL * BAUD_RATE)))
12
13 static void usart0_init(void)
14 {
15     /* Set TX pin as output, RX pin as input */
16     /* Adjust pins according to your board routing */
17     PORTA.DIRSET = PIN0_bm; /* Example TX */
18     PORTA.DIRCLR = PIN1_bm; /* Example RX */
19 }
```

7. Basic USART Initialization (Polling Mode)

```
12
13 static void usart0_init(void)
14 {
15     /* Set TX pin as output, RX pin as input */
16     /* Adjust pins according to your board routing */
17     PORTA.DIRSET = PIN0_bm; /* Example TX */
18     PORTA.DIRCLR = PIN1_bm; /* Example RX */
19
20     /* Set baud rate */
21     USART0.BAUD = USART0_BAUD_VALUE;
22
23     /* 8 data bits, no parity, 1 stop bit (default) */
24     USART0.CTRLA = USART_CMODE_ASYNCHRONOUS_gc /* Asynchronous Mode */
25         | USART_CHSIZE_8BIT_gc /* Character size: 8 bit */
26         | USART_PMODE_DISABLED_gc /* No Parity */
27         | USART_SBMODE_1BIT_gc; /* 1 stop bit */
28
29     /* Enable TX and RX */
30     USART0.CTRLB = USART_TXEN_bm | USART_RXEN_bm;
31 }
```

Important:

- Adjust TX/RX pins according to your Curiosity Nano routing.
- Many boards route USART to the virtual COM port automatically.



8. Transmitting One Byte (Polling)

8. Transmitting One Byte (Polling)

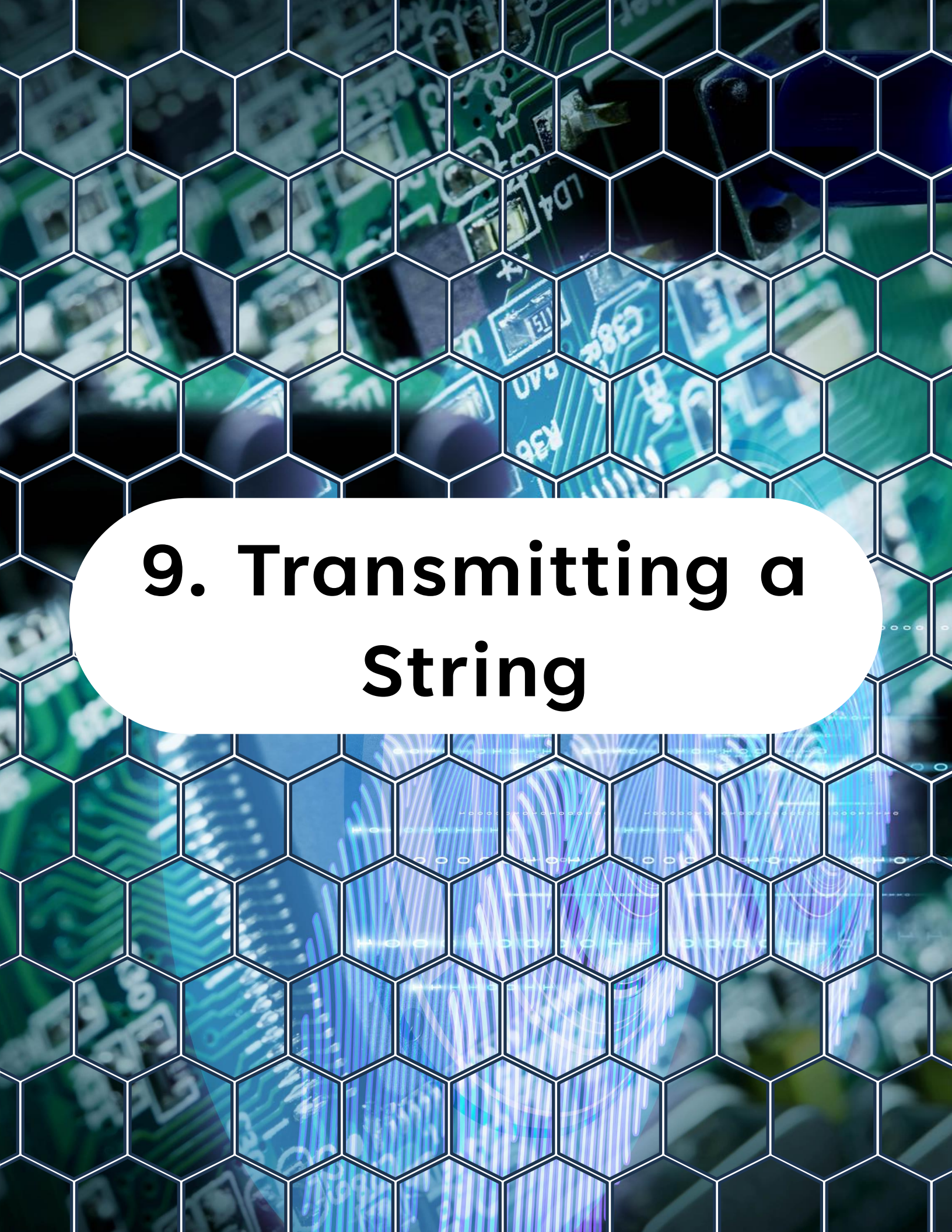
To transmit:

- Wait until data register empty
- Write byte to transmit register

```
1 static void usart0_write_byte(uint8_t data)
2 {
3     // Wait until transmit buffer is empty and
4     // ready to accept new data
5     while (!(USART0.STATUS & USART_DREIF_bm)) {
6         ; // Busy-wait for data register empty flag
7     }
8
9     // Load data into transmit register
10    USART0.TXDATA = data;
11
12    // Wait for the last byte to be completely transmitted.
13    while (!(USART0.STATUS & USART_TXCIF_bm)) {
14        ; // Busy-wait for transmission complete flag
15    }
16 }
```

DREIF = Data Register Empty Interrupt Flag

We use it here as a status flag.



9. Transmitting a String

9. Transmitting a String

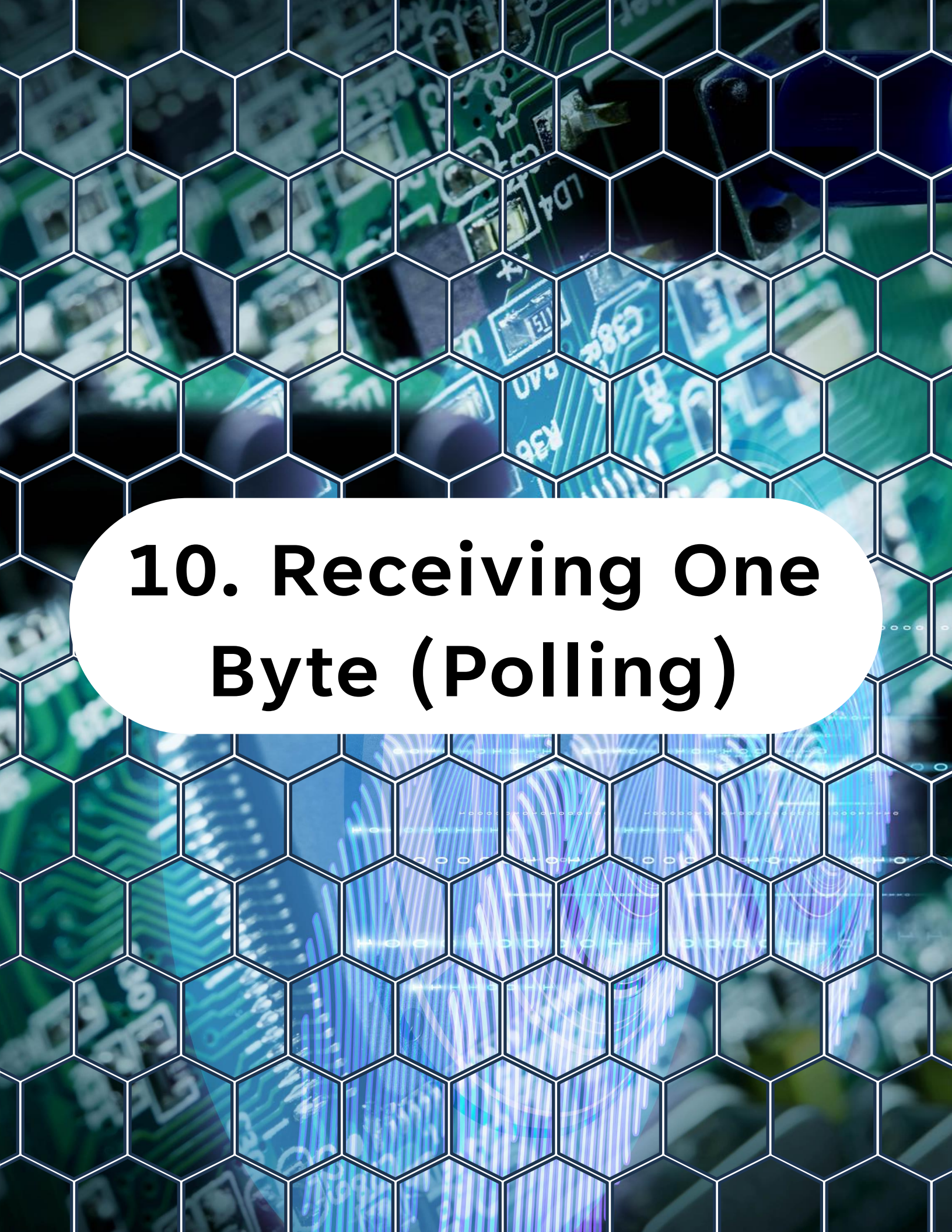
Write a string to USART0.

This function takes a null-terminated string and sends each character to **USART0** using the `usart0_write_byte` function.



```
1 static void usart0_write_string(const char *str)
2 {
3     while (*str)
4     {
5         // Write the current character to USART0
6         usart0_write_byte((uint8_t)(*str));
7         str++;
8     }
9 }
```

Now you can send readable messages to a PC terminal.



10. Receiving One Byte (Polling)

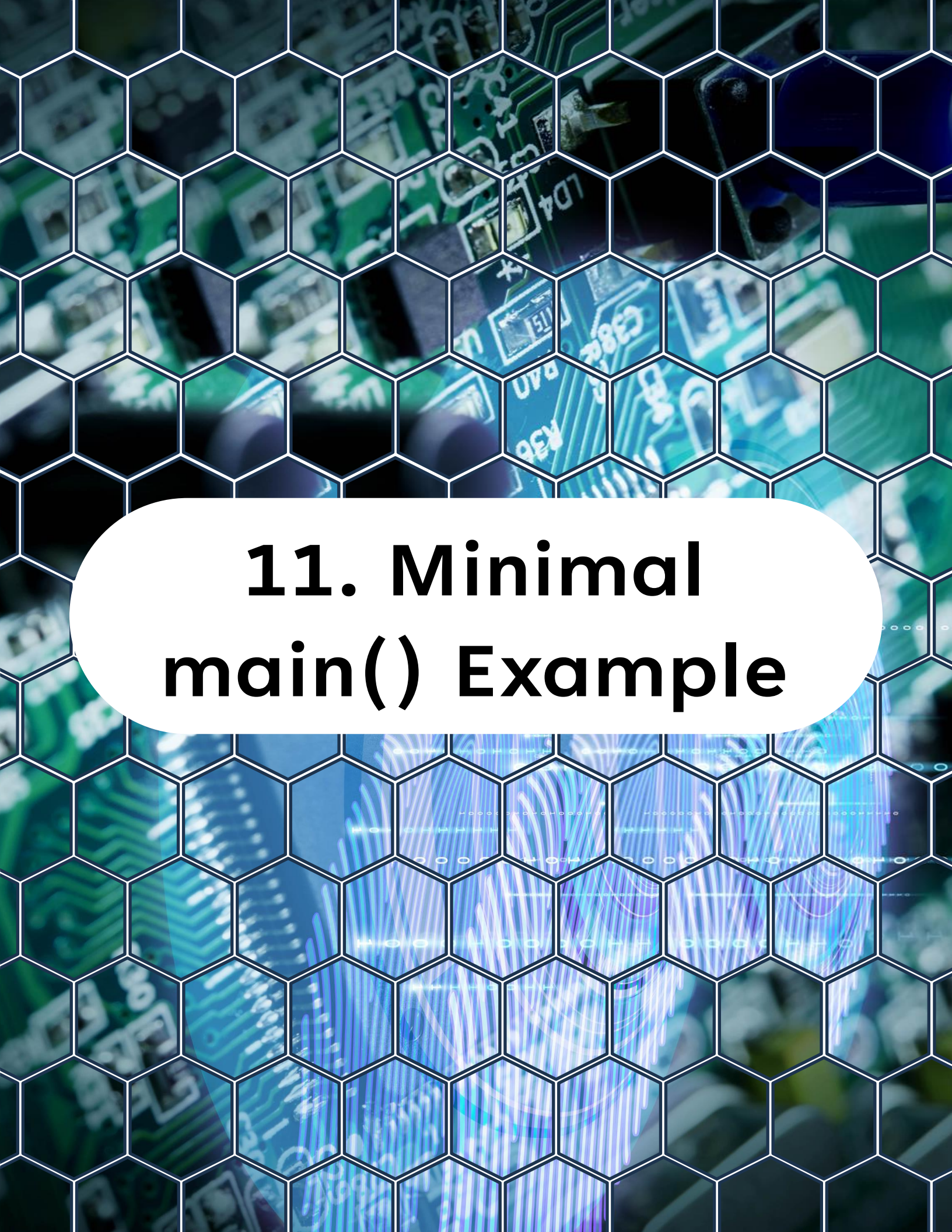
10. Receiving One Byte (Polling)

To receive:

- Wait until RX complete flag set
- Read received data

```
1 static uint8_t usart0_read_byte(void)
2 {
3     // Wait until data is received.
4     while (!(USART0.STATUS & USART_RXCIF_bm))
5     {
6         /* Wait */
7     }
8
9     return USART0.RXDATAL;
10 }
```

The **RXCIF** (Receive Complete Interrupt Flag) bit in the **STATUS** register indicates that there is unread data in the receive buffer. We will wait until this bit is set before reading the data from the **RXDATAL** register.



11. Minimal main() Example

11. Minimal main()

Example

```
1  int main(void)
2  {
3      char received_char;
4
5      // Initialize USART0
6      USART0_init();
7
8      // Small delay to ensure USART is ready
9      _delay_ms(100);
10
11     // Send welcome message
12     usart0_write_string("\r\n");
13     usart0_write_string("=====\r\n");
14     usart0_write_string("AVR128DA48 USART0 Polling Demo\r\n");
15     usart0_write_string("=====\r\n");
16     usart0_write_string("Echo Mode Active\r\n");
17     usart0_write_string("Type any character to echo it back\r\n");
18     usart0_write_string("-----\r\n\r\n");
19
20     // Main loop - Echo received characters
21     while (1)
22     {
23         // Wait for and receive a character
24         received_char = usart0_read_byte();
25
26         // Echo the character back
27         usart0_write_byte(received_char);
28
29         // Optional: Add line feed if carriage return is received
30         if (received_char == '\r')
31         {
32             usart0_write_byte('\n');
33         }
34     }
35
36     return 0; // Never reached
```

11. Minimal main()

Example

```
28
29     // Optional: Add line feed if carriage return is received
30     if (received_char == '\r')
31     {
32         usart0_write_byte('\n');
33     }
34 }
35
36 return 0; // Never reached
37 }
```

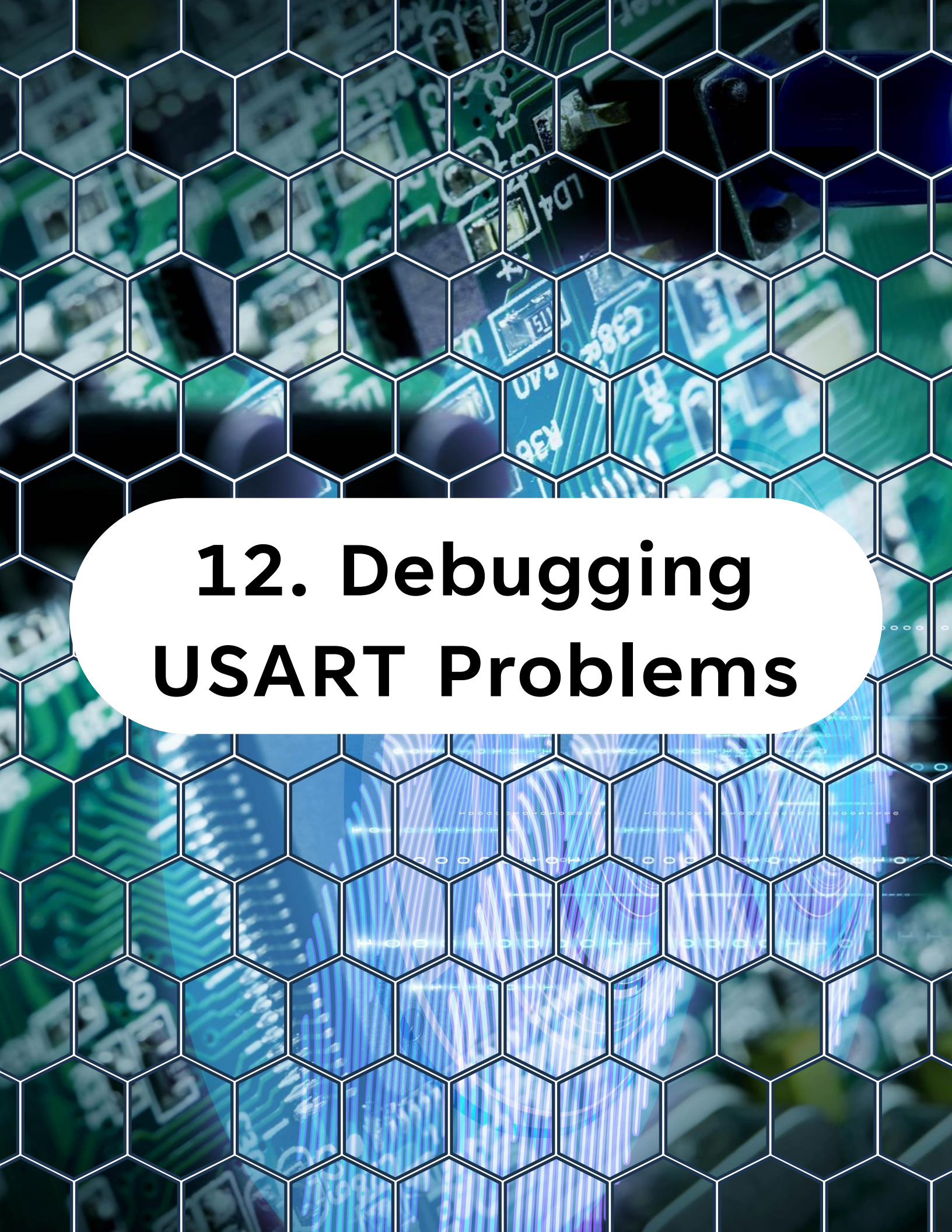
What this does:

- Sends startup message
- Echoes any received character
- Demonstrates bidirectional communication

Open a serial terminal:

- Set baud to 9600
- 8N1
- Correct COM port

Type something. You should see it echoed back.



12. Debugging USART Problems

12. Debugging USART Problems

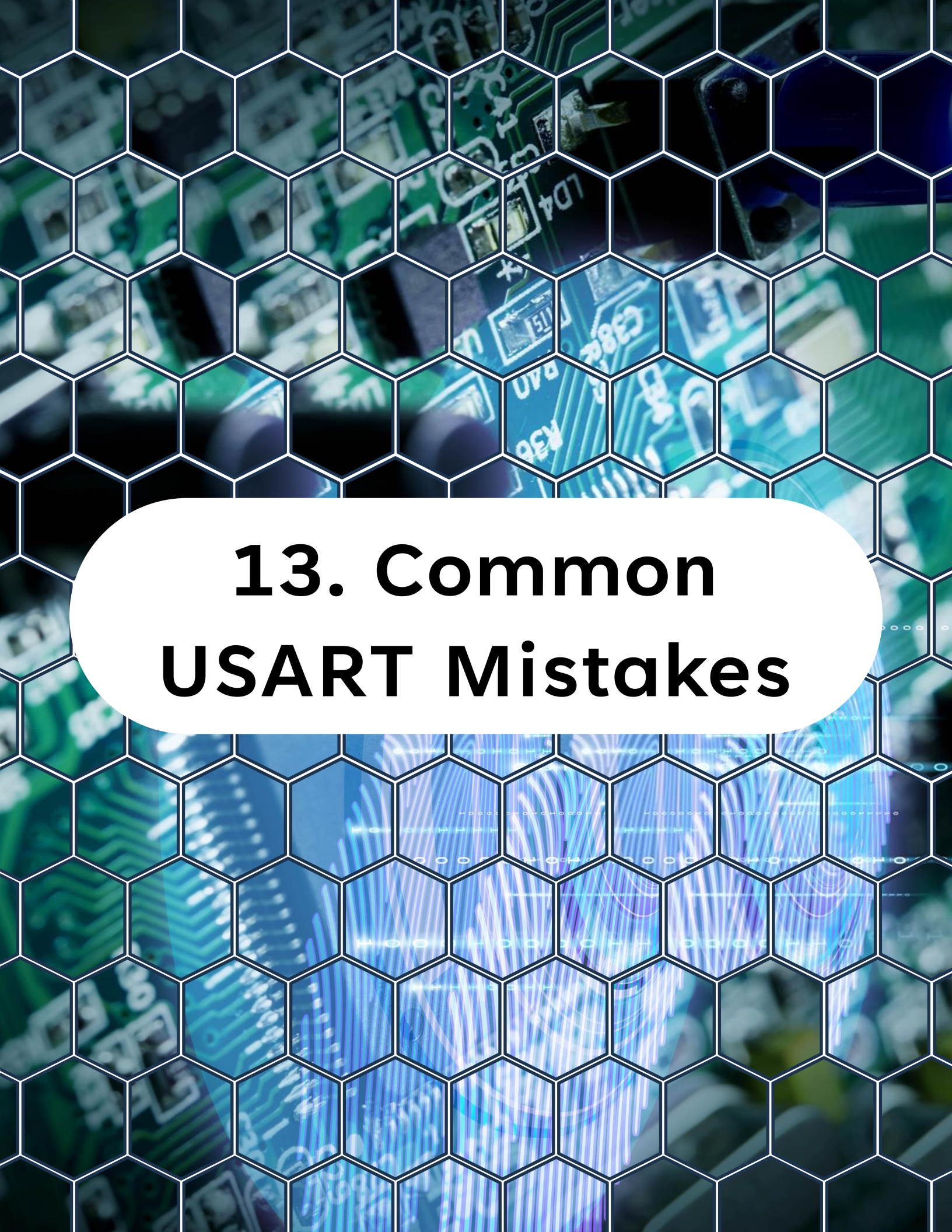
If nothing appears in terminal:

- Check baud rate setting
- Verify **F_CPU** is correct
- Confirm **TX/RX** pin routing
- Confirm correct COM port selected
- Confirm terminal settings (8N1, no flow control)

If you see garbage characters:

- Baud rate mismatch
- Wrong clock configuration
- Wrong BAUD register calculation

Serial communication is very unforgiving about timing.



13. Common USART Mistakes

13. Common USART Mistakes

- Forgetting to enable **TX** or **RX**
- Not waiting for **DREIF** before writing
- Ignoring **RXCIF** before reading
- Incorrect baud formula
- Wrong pin configuration

Fix these systematically. Do not guess.



14. What You Learned Today

14. What You Learned Today

By the end of **Day 11**, you understand:

- How asynchronous serial communication works
- How USART generates baud rate
- How to configure USART registers directly
- How to transmit and receive data using polling
- How to debug UART communication problems

Your MCU can now communicate with a PC.



15. What Comes Next

15. What Comes Next

Day 12 - USART Advanced Usage

Next, you will:

- Use interrupts for RX and TX
- Build a ring buffer
- Avoid blocking waits
- Create a non-blocking command interface

This is where USART becomes production-ready instead of demo-level.

"Thanks for watching!

If you enjoyed this video, make sure to hit the like button and subscribe to stay updated with my latest content.

Don't forget to check out my other videos for more tips and tutorials on Embedded C, Python, hardware designs, etc. Keep exploring, keep learning, and I'll see you in the next video!"

<https://www.linkedin.com/in/yamil-garcia>

<https://www.youtube.com/@LearningByTutorials>

<https://github.com/god233012yamil>